File Transfer Protocol: Two Connections, Active vs Passive Mode, ASCII vs Binary, Resume, FTPS, and Bounce Protection

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 Commands and Replies **4**

3 System Architecture **5**

4 Implementation **5**

4.1 Active and Passive Data Connections 5

4.2 ASCII vs Binary 6

4.3 Abort and Resume 7

4.4 Security: Jail, Anonymous Access and FTPS 7

5 Experiments, Verification and Results **7**

5.1 Running the Module 7

5.2 Active vs Passive Behind a Firewall 7

5.3 Automated Verification 8

5.4 Observations 8

6 Conclusion **9**

AI Acknowledgement **9**

1 Introduction

The **File Transfer Protocol** (**FTP**, **RFC 959**) copies files between a client and a server. It is older than the web and has one design decision that makes it unique among the common application protocols: it uses **two TCP connections**.

- The **control connection** (server port 21) stays open for the whole session and carries text commands (`USER`, `RETR`, ...) and three-digit replies. This is called *out-of-band* control.
- A separate **data connection** is opened for every file transfer or directory listing and closed at its end; the close marks the end of the file.

Who opens the data connection depends on the mode. In **active mode** (`PORT`) the *server* connects back to a port the client announced, which client-side firewalls and NAT routers block. In **passive mode** (`PASV`) the *client* connects to a port the server opened, which is why every modern client defaults to it. FTP also distinguishes **ASCII** (`TYPE A`) transfers, which convert line endings, from **binary** (`TYPE I`) transfers, which copy bytes exactly. Like Telnet, plain FTP sends the password in clear text; **FTPS** (RFC 4217) adds TLS.

1.1 What Was Built

This module implements an FTP server and client from the RFCs in pure Java 11 with no external libraries:

1. **Server**: RFC 959 commands, active mode with `PORT`/`EPRT` (the server connects from port 20-style *control port − 1*), passive mode with `PASV`/`EPSV` (RFC 2428), `TYPE A` and `TYPE I`, `REST` and `ABOR`, `SIZE`, `MDTM` and `FEAT` (RFC 3659), a per-user home directory jail, anonymous read-only access, and **FTPS** with `AUTH TLS`, `PBSZ` and `PROT P`.
2. **Security**: **bounce-attack protection** (RFC 2577) and salted SHA-256 password hashes.
3. **Client**: the same modes, with an optional simulated client firewall that blocks incoming connections.
4. **Wire tap** on the control connection, showing the password in plain FTP and only TLS records in FTPS.
5. **Dashboard** on `http://localhost:8086` that draws the two connections as a signal wire (control) and a track (data) and steps through every command and reply.

Table 1: Ports and accounts

Item	Value
Control connection	TCP 2121 (also the real port 21 when it is free)
Active-mode data	from TCP 2120 (control port − 1, the analogue of port 20)
Passive-mode data	TCP 50000–50100
Wire tap	TCP 2122 → 2121
Dashboard	HTTP 8086
Users	`alice/alice123`, `bob/bob123`, `subham/ju2026`, `anonymous` (read-only)

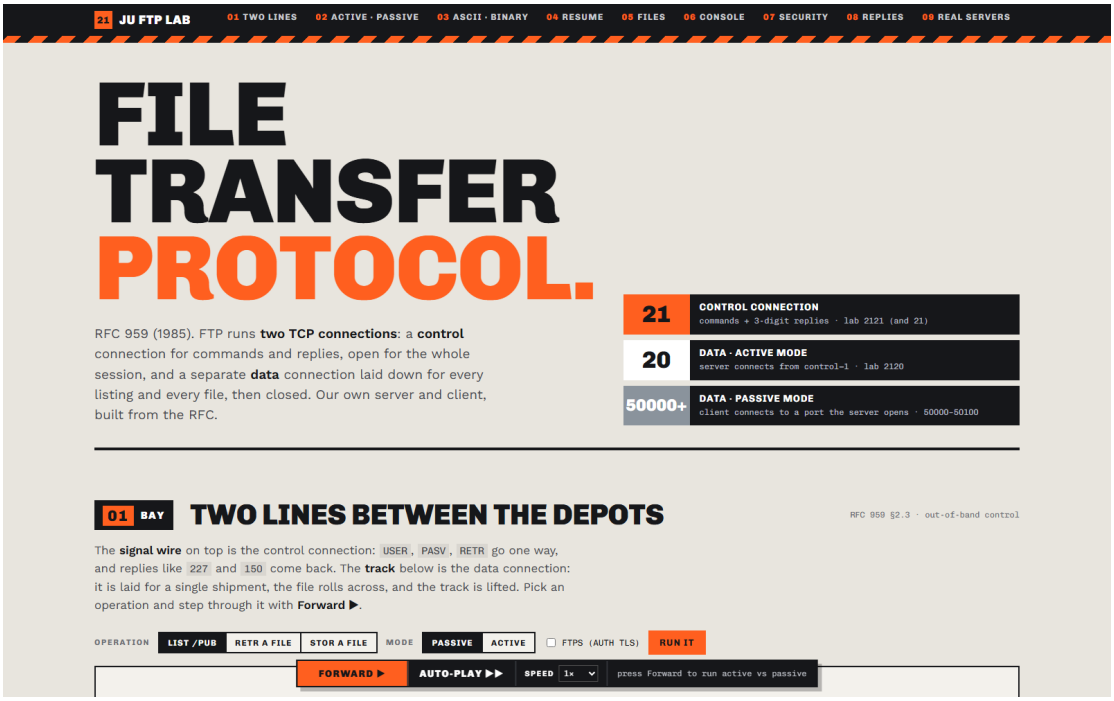


Figure 1: The FTP dashboard.

2 Commands and Replies

Commands are ASCII lines ending in `CRLF`. Every reply begins with three digits; the first gives the outcome and the second the subject.

Table 2: Meaning of the reply-code digits

1st	Outcome	2nd	Subject
1	Preliminary: started, wait for another reply	0	Syntax
2	Completed	1	Information
3	Intermediate: send the next command	2	Connections
4	Transient failure: try again later	3	Authentication
5	Permanent failure	5	File system

```
   TCP connection 57026 -> 127.0.0.1:2121        the CONTROL connection, kept for the whole
    session
S: 220 JU-FTP ready . RFC 959 . ftp.cse.myuniv.edu
C: USER alice
S: 331 Password required for alice
C: PASS ********                                  travels in PLAINTEXT unless AUTH TLS came
    first
S: 230 User alice logged in, home /home/alice
C: TYPE I
S: 200 TYPE set to I (binary: bytes sent exactly as stored)
C: PASV
S: 227 Entering Passive Mode (127,0,0,1,195,80)   port = 195 x 256 + 80 = 50000
   DATA connection: client 57027 -> server 50000 (the client opens it)
C: RETR /pub/rfc959-notes.txt
S: 150 Opening BINARY mode data connection for /pub/rfc959-notes.txt (231 bytes)
   DATA connection closed . 231 bytes             end of file = end of the data connection
S: 226 Transfer complete: 231 bytes in 5 ms
```

```
C: QUIT
S: 221 Goodbye
```

Listing 1: A passive-mode download, as recorded by the dashboard (C = client, S = server).

3 System Architecture

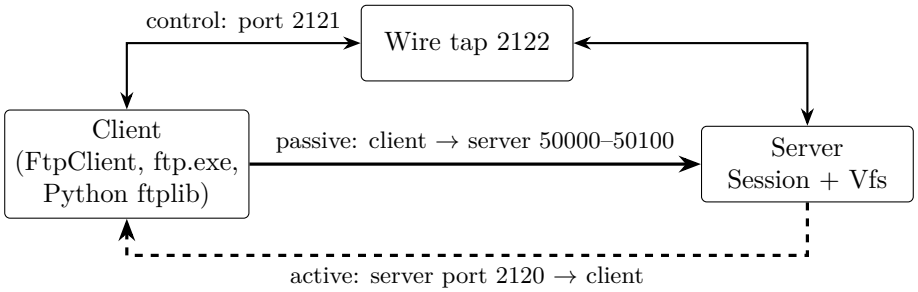


Figure 2: One control connection, and a new data connection per transfer opened by either side.

Table 3: Source files

File	Responsibility
`server/FtpServer.java`	Listener, users and hashes, passive port range, settings (anonymous, slow link).
`server/Session.java`	One control connection: command dispatch, data connections, transfers, TLS.
`server/Vfs.java`	Virtual file system with a home-directory jail per user.
`server/Codes.java`	Reply-code meanings and the ASCII line-ending conversion.
`client/FtpClient.java`	Client with active/passive mode, firewall simulation and FTPS.
`tap/Tap.java`	Control-connection sniffer.
`dashboard/DashboardServer.java`	HTTP API and step-by-step traces.

4 Implementation

4.1 Active and Passive Data Connections

For `PORT h1,h2,h3,h4,p1,p2` the server records the client's address and port ($p_1 \times 256 + p_2$) and connects there at the next transfer, binding its own end to *control port − 1*. For `PASV` it opens a listening socket from its passive range and announces it in the same six-number format; `EPSV` announces only the port (`229 (|||port|)`), which also works for IPv6.

Bounce-attack protection. RFC 959 allows `PORT` to name *any* host, so an attacker could make the server open connections to a third machine on its behalf (the FTP bounce attack, RFC 2577). The server only connects back to the address of the control connection and never to a privileged port.

```
1 private void port(String a) {
2     String[] p = a.split(",");
3     if (p.length != 6) { reply("501 PORT h1,h2,h3,h4,p1,p2"); return; }
4     activeTo(p[0] + "." + p[1] + "." + p[2] + "." + p[3],
5              Integer.parseInt(p[4].trim()) * 256 + Integer.parseInt(p[5].trim()), "PORT");
```

```
}
private void activeTo(String ip, int port, String how) {
    // RFC 2577: only connect back to the client itself, never to a privileged port
    if (!ip.equals(ctrl.getInetAddress().getHostAddress()) || port < 1024) {
        reply("500 Illegal " + how + " command: " + ip + ":" + port + " (bounce-attack protection, RFC 2577)");
        return;
    }
    closePasv();
    active = new InetSocketAddress(ip, port);
    reply("200 " + how + " ok: I will connect to " + ip + ":" + port + " for the next transfer (active mode)");
}
private void pasv(boolean extended) throws IOException {
    closePasv(); active = null;
    pasv = srv.pasvSocket(ctrl.getLocalAddress());          // a port from 50000-50100
    int p = pasv.getLocalPort();
    byte[] a = ctrl.getLocalAddress().getAddress();
    if (extended) reply("229 Entering Extended Passive Mode (|||" + p + "|)");
    else reply(String.format("227 Entering Passive Mode (%d,%d,%d,%d,%d,%d)",
            a[0] & 0xFF, a[1] & 0xFF, a[2] & 0xFF, a[3] & 0xFF, p / 256, p % 256));
}
```

Listing 2: PORT, PASV and bounce protection in `server/Session.java`.

```
private Socket openData() throws IOException {
    Socket s;
    if (pasv != null) {
        pasv.setSoTimeout(10_000);
        try { s = pasv.accept(); } finally { closePasv(); }      // passive: the CLIENT connects
    } else if (active != null) {
        InetSocketAddress to = active; active = null;
        s = new Socket(); s.setReuseAddress(true);
        int src = ctrl.getLocalPort() - 1;                       // like port 20 next to port 21
        try { s.bind(new InetSocketAddress(ctrl.getLocalAddress(), src)); }
        catch (BindException e) { s.close(); s = new Socket(); }
        s.connect(to, 4000);                                     // active: the SERVER connects
    }   // (a transfer without PORT or PASV is refused earlier with "425 Use PORT or PASV first")
    return s;
}
```

Listing 3: Opening the data connection: accept in passive mode, connect from port control-1 in active mode.

4.2 ASCII vs Binary

In ASCII mode the data is sent as NVT-ASCII: every line ends in `CRLF` on the wire, whatever the local convention. A Unix text file therefore grows by one byte per line. Applied to an image, the same conversion inserts a `0D` before every `0A` byte and corrupts it.

```
/** ASCII mode on the wire: every LF becomes CRLF (RFC 959 NVT-ASCII). */
public static byte[] toCrlf(byte[] d) {
    ByteArrayOutputStream b = new ByteArrayOutputStream(d.length + d.length / 16);
    int prev = -1;
    for (byte x : d) { int c = x & 0xFF; if (c == '\n' && prev != '\r') b.write('\r'); b.write(c); prev = c; }
    return b.toByteArray();
```

```
}
```

Listing 4: ASCII mode on the wire (`server/Codes.java`).

4.3 Abort and Resume

`ABOR` on the control connection stops a transfer in progress: the server closes the data connection and replies `426` (transfer aborted) followed by `226` (abort successful). `REST n` before `RETR` makes the server skip the first n bytes, so an interrupted download continues where it stopped; the dashboard verifies that the resumed file is byte-identical to the original. HTTP's `Range` header solves the same problem in the HTTP module.

4.4 Security: Jail, Anonymous Access and FTPS

Each user is confined to a home directory: `CWD ../../..` stays at `/`, and `MKD` outside the home directory returns `550`. Anonymous users see only `/pub` and cannot store files (`553`). For FTPS the client sends `AUTH TLS`; after `234` both ends perform a TLS handshake on the control connection, then `PBSZ 0` and `PROT P` extend protection to the data connections. The wire tap shows `PASS alice123` on a plain session and only TLS records after `AUTH TLS`.

5 Experiments, Verification and Results

5.1 Running the Module

```
cd FTP
run.bat                  # server, wire tap and dashboard (http://localhost:8086)
run.bat --verify         # 32 automated checks
python interop_test.py   # Python ftplib (plain + FTPS) and Windows ftp.exe (10 checks)

ftp 127.0.0.1            # Windows ftp.exe uses active mode: user alice, then binary, get /
    pub/logo.png
```

Listing 5: Starting the server, running the checks, and using real clients.

5.2 Active vs Passive Behind a Firewall

The dashboard runs `LIST /pub` in both modes side by side, with and without a simulated client firewall that drops incoming connections.

Table 4: Observed results

	Active (PORT)	**Passive (PASV)**
No firewall	`200 PORT ok`; the server connects from port 2120 to the client; `226`, listing received	`227`; the client connects to port 5000x; `226`, listing received
Client firewall	The server's incoming connection is dropped: **`425 Can't open data connection`**	The client's outgoing connection passes: listing received

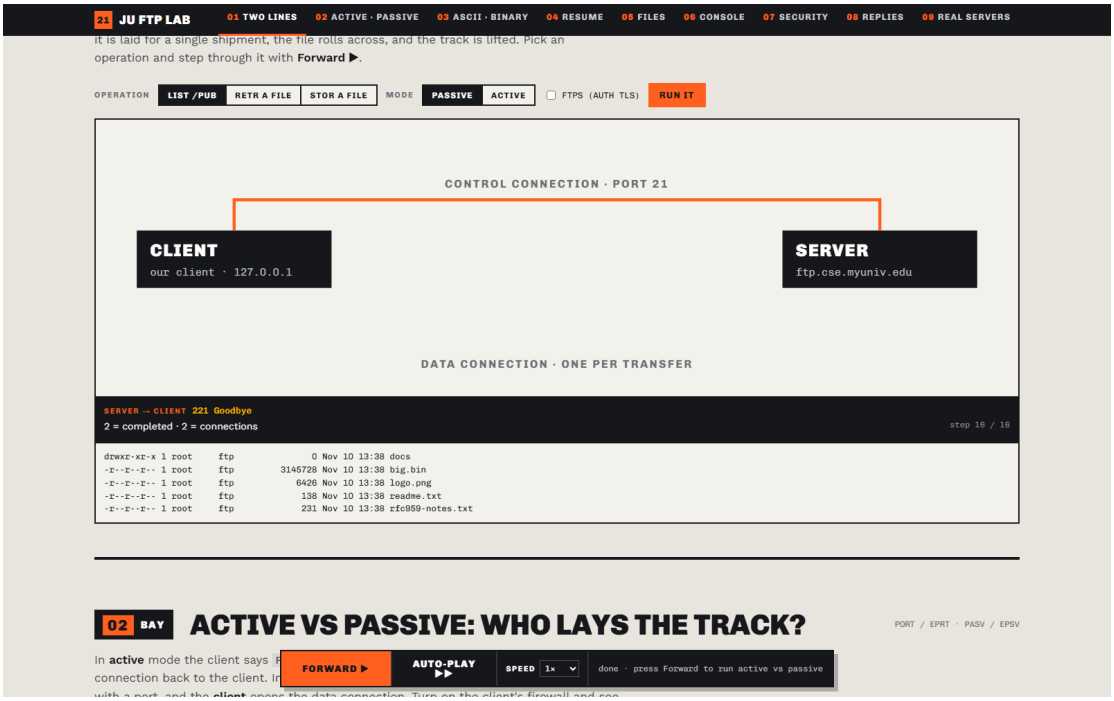


Figure 3: The two connections drawn as a signal wire (control) and a track (data).

5.3 Automated Verification

All **32 checks** of `Verify.java` and all **10** interoperation checks pass.

Table 5: Selected verification checks (all passed)

220 greeting; `USER` → 331; wrong `PASS` → 530; right `PASS` → 230
`FEAT`: multi-line 211 reply lists SIZE, REST, EPSV, AUTH TLS; unknown command → 502
`CWD ../../..` cannot escape the jail; `MKD` outside the home folder → 550
Passive LIST (client connects); active LIST (server connects from port control−1)
Active behind a client firewall → 425; passive behind the same firewall still works
`EPSV` → 229 (|||port|); `PORT` to another host → 500 (bounce-attack protection)
`RETR` and `STOR` in binary: byte-identical image; the same image uploaded in ASCII mode is corrupted
ASCII `RETR` of a text file: every LF becomes CR LF on the wire
`SIZE` and `MDTM` → 213; `RNFR`/`RNTO`/`DELE`
`ABOR` half-way: 426 then 226; `REST` resumes and the file is byte-identical
Anonymous login sees `/pub`; anonymous `STOR` → 553
FTPS: `AUTH TLS` + `PBSZ 0` + `PROT P`, then an encrypted `RETR`
The wire tap reads the password on plain FTP and sees only TLS records after `AUTH TLS`

5.4 Observations

1. **Two connections make the firewall problem.** Active mode failed with 425 as soon as the client refused incoming connections, while passive mode worked; this is the practical reason for passive mode and EPSV.

2. **Transfer type matters.** The ASCII-mode copy of a PNG was rejected by the image decoder, while the binary copy had the same SHA-256 hash as the original.

3. **Standard clients work.** Python's `ftplib` (plain and FTPS) and Windows `ftp.exe` (which uses active mode) completed logins, listings and transfers against the server.

6 Conclusion

1. **Out-of-band control is FTP's defining feature.** Keeping commands on one long-lived connection and data on short per-transfer connections makes aborting and resuming simple, but it doubles the number of connections a firewall must allow.

2. **Passive mode exists because of firewalls and NAT.** The experiment reproduced the 425 failure of active mode behind a client firewall and showed passive mode succeeding.

3. **FTP needs both protection and encryption.** Restricting `PORT` to the client's own address blocks the bounce attack, the jail confines users, and `AUTH TLS` removes the plaintext password that the wire tap otherwise recovers.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.